Worksheet 2 — Secure SDLC & Tooling (3 hrs)

> **Course:** Software Security (KOSEN69) · **Week 2 Aligned to:** OWASP 2025 (A05 Injection [CWE-89, CWE-78], A04 Cryptographic Failures [CWE-327], A02 Security Misconfiguration [CWE-798, CWE-489]) · CWE-798, CWE-89, CWE-78, CWE-327, CWE-489 **Signature game:** "Bug Triage Race" (scan → triage; score = true positives − misclassified)

> **Ethics note:** The scanners run only against the provided `vulnerable-repo/` on your own machine. Do not point SAST/secret scanners at third-party repos or production systems without authorization. Treat any secret you find here as fake lab data.

Part 1 — Student Information

Name	Student ID	Date	Group

Part 2 — Lecture Questions

Answer in your own words (2–4 sentences each).

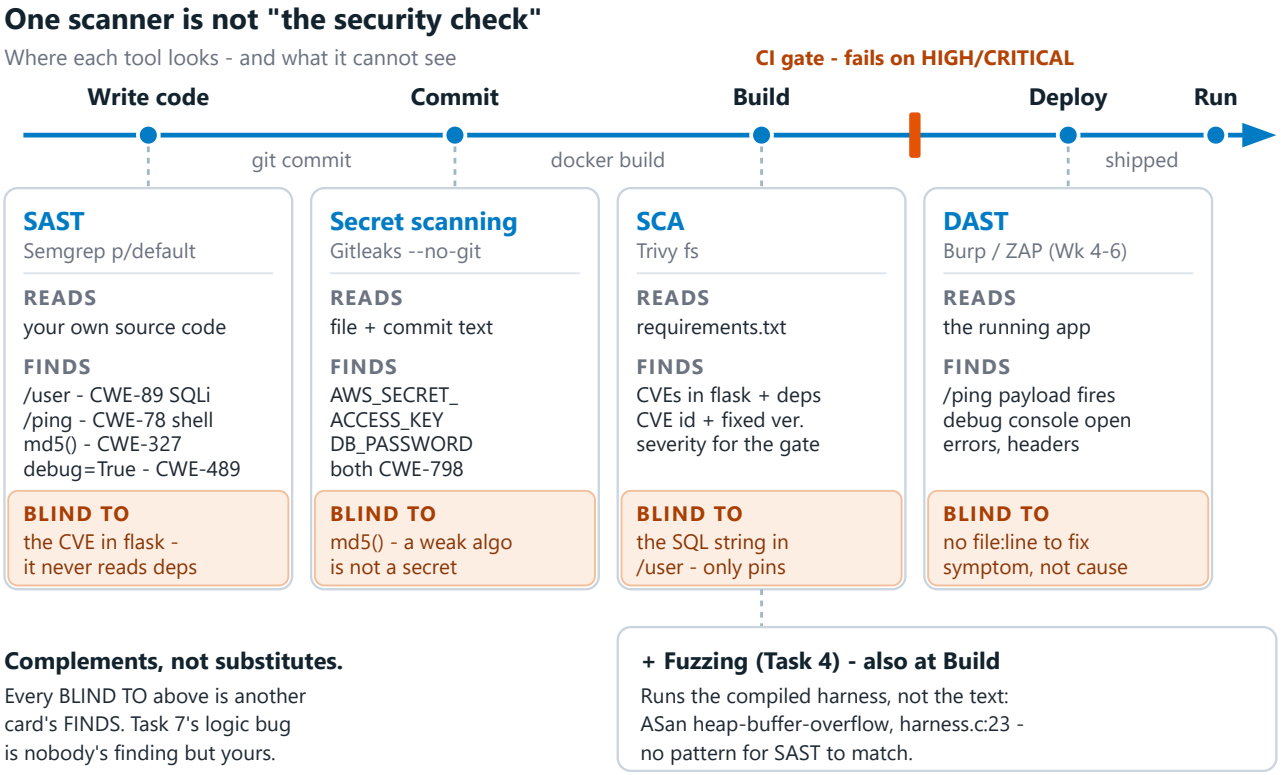
1. Distinguish SAST, DAST, and SCA — what does each see, and when in the SDLC does each run?
2. What is secret scanning, and why do hardcoded secrets keep ending up in repos?
3. What does "shift-left / DevSecOps" mean in practice for a CI pipeline?
4. Why is coverage-guided fuzzing considered the dominant modern bug-finding technique?
5. Define true positive vs. false positive in scanner triage, and why misclassifying both directions is costly.

Answers

1. SAST analyzes source/bytecode without executing it — it runs at write/commit time and catches insecure *patterns* (string-built SQL, `shell=True`) but has no runtime context, so it misses business-logic flaws and produces some false positives. DAST attacks a *running* app from the outside, like a black-box pentest — it runs at staging/pre-release and proves a flaw is actually reachable over HTTP, but it can't point at the line of code. SCA inspects your dependency manifest for known-vulnerable third-party versions — it runs at build time and catches supply-chain CVEs (e.g. an outdated PyJWT) that neither SAST nor DAST would ever flag, since the vulnerable code isn't yours.
2. Secret scanning is regex/entropy-based detection of credential-shaped strings (API keys, passwords, tokens) committed into a repo, including its git history. Hardcoded secrets keep happening because pasting a working key inline is faster under deadline pressure than wiring up a secrets manager or env vars — and once committed, the value lives in git history essentially forever, even after later deletion from the working tree.
3. It means moving security checks into the same pipeline developers already use before merge — pre-commit hooks and CI gates on every PR — instead of a one-time pentest right before release. A SQL injection gets caught and fixed by the author in minutes at PR time, instead of being discovered by a security team (or an attacker) in production months later.
4. It needs no human-written test cases or input model — it mutates inputs and uses code-coverage feedback to steer toward unexplored paths, running millions of cheap iterations. That's how it found the `harness.c` heap-buffer-overflow in under a second (Task 4): the bug only exists for one specific byte

sequence, which is exactly the kind of runtime-only, input-dependent flaw a pattern-matching SAST pass over the same 4 lines cannot see.

5. A true positive is a finding that's a real, exploitable flaw in context (the SQLi at `app.py:19` genuinely lets you manipulate the query). A false positive is a flagged pattern that isn't a real risk in context (e.g., a weak-hash call in code nothing ever invokes — see Task 3). Misclassifying a TP as FP ships a real vulnerability to production; misclassifying an FP as TP wastes remediation effort and, repeated often enough, trains developers to ignore the scanner entirely (alert fatigue), which is just as dangerous as missing the bug outright.



Part 3 — Hands-on Lab (180 min)

Learning goals: run a SAST tool and a secret scanner, triage findings by CWE/severity, and remediate real flaws. **Prerequisites:** Docker installed; internet to pull the Semgrep/Gitleaks images.

Environment setup

```
cd labs/week02-sdlc-tooling
cat scan.sh                # see exactly what it runs
bash scan.sh               # Semgrep (p/default + p/owasp-top-ten) then Gitleaks on ./vulnerable-repo
```

Target under scan: `vulnerable-repo/app.py` (plus `requirements.txt`). It contains five planted flaws.

What to submit per task: the command/payload run + a screenshot of the finding + a 2–3 sentence mitigation.

Task 0 — Onboarding (5 min) · *Goal:* confirm tooling. *Steps:* run `bash scan.sh`; confirm both Semgrep and Gitleaks sections produce output. *Deliverable:* screenshot showing both tools ran.

> **Result:** Ran `bash scan.sh`. On Windows/Git Bash this needed `MSYS_NO_PATHCONV=1 bash scan.sh` — without it, Git Bash rewrites the `/src` and `/repo` Docker mount paths into a Windows path (e.g. `C:/Program Files/Git/src`) before Docker ever sees them, and both tools fail (`Invalid scanning root`, `stat ...: no such file or directory`). With that set, both sections produced output: Semgrep → `Ran 322 rules on 2 files: 10 findings.`; Gitleaks → `leaks found: 2`. *(Screenshot with your identity stamp is still on you.)*

Task 1 — SAST sweep with Semgrep (25 min) · *Goal:* find code flaws. *Steps:* read the Semgrep output; locate the SQL injection in `/user` (CWE-89, string-formatted query), the OS command injection in `/ping` (CWE-78, `shell=True`), the weak `md5` password hash (CWE-327), and `debug=True` (CWE-489). *Deliverable:* one screenshot per finding with the file:line.

> **Result:**
>
> | Flaw | CWE | File:Line | Rule |
> |---|---|---|---|
> | SQL injection in `/user` | CWE-89 | `app.py:19-20` | `formatted-sql-query` + `tainted-sql-string` + `sql-injection-db-cursor-execute` |
> | OS command injection in `/ping` | CWE-78 | `app.py:26` | `subprocess-injection` + `dangerous-subprocess-use` + `subprocess-shell-true` |
> | Weak MD5 password hash | CWE-327 | `app.py:30` | `insecure-hash-algorithm-md5` |
> | `debug=True` | CWE-489 | `app.py:33` | `flask.security.audit.debug-enabled` |
>
> 10 total findings because several rules independently matched the same 2 real bugs (SQLi and command injection each tripped 3 overlapping rules).

Task 2 — Secret scan with Gitleaks (15 min) · *Goal:* find leaked credentials. *Steps:* read the Gitleaks output; identify `AWS_SECRET_ACCESS_KEY` and `DB_PASSWORD` (CWE-798). *Deliverable:* screenshot + the rule that fired for each.

> **Result:**
>
> | Secret | File:Line | Rule |
> |---|---|---|
> | `AWS_SECRET_ACCESS_KEY` | `app.py:11` | `generic-api-key` |
> | `DB_PASSWORD` | `app.py:12` | `generic-api-key` |

Task 3 — Bug Triage Race (30 min) · *Goal:* triage accurately. *Steps:* build a table with columns *Tool* | *File:Line* | *CWE* | *Severity* | *TP/FP* | *Fix idea*; mark at least 3 true positives and 1 likely false positive and justify each. (Score = TP − misclassified.) *Deliverable:* the completed triage table.

> **Result:**

Tool	File:Line	CWE	Severity	TP/FP	Fix idea
Semgrep	`app.py:19-20`	CWE-89	Critical	**TP**	Parameterized query: `execute("... WHERE name = ?", (name,))`
Semgrep	`app.py:26`	CWE-78	Critical	**TP**	Drop `shell=True`; pass an argument list: `["ping","-c","1",host]`
Gitleaks	`app.py:11-12`	CWE-798	High	**TP**	Move both secrets to environment variables / a secrets manager
Semgrep	`app.py:33`	CWE-489	Medium	**TP**	`app.run(debug=False)` in anything that isn't local dev
Semgrep	`app.py:30`	CWE-327	Low (in this file)	**Likely FP**	See justification

FP justification: `store_password()` at line 30 is a real weak-hash pattern (MD5), but grepping the file shows it is never called by any route — it's dead code. The finding is technically correct (MD5 is insecure) but *not currently exploitable*, since no request path reaches it. That's the distinction this exercise is testing: "bad pattern present" is not the same as "exploitable now," even though it's still worth fixing before anything wires it up.

Task 4 — Fuzzing intro (10 min) · *Goal:* see coverage-guided fuzzing find a bug SAST won't. *Steps:* in the `labs/toolbox` container (Apple clang has no libFuzzer runtime), build `clang -g -fsanitize=address,fuzzer harness.c -o fuzz`, then **seed the corpus** and run it: `mkdir -p corpus && printf 'FUZ' > corpus/seed && ./fuzz corpus`. It crashes almost immediately with an AddressSanitizer heap-buffer-overflow at `harness.c:23` (the `data[3]` read with no `size > 3` check). Seeding matters: an unseeded `./fuzz` has to rediscover the magic bytes by chance and often finds nothing for minutes — that unpredictability is itself worth a sentence in your write-up. (The deep fuzzing+exploit lab is Week 11.) *Deliverable:* the ASan crash output (or a screenshot) + a 2-sentence note on why fuzzing finds this bug when a linter/SAST pass over the same 4-line check would not.

Result: Built inside `labs/toolbox` (`silkeh/clang:19`, which has a working libFuzzer runtime — host/Apple clang doesn't):

```
docker build -t softsec-toolbox labs/toolbox
docker run --rm --cap-add=SYS_PTRACE --security-opt seccomp=unconfined -v
"$PWD":/work -w /work softsec-toolbox \
  bash -c "clang -g -fsanitize=address,fuzzer harness.c -o fuzz && mkdir -p
corpus && printf 'FUZ' > corpus/seed && ./fuzz corpus"
```

Crashed immediately on the seeded corpus:

```
SUMMARY: AddressSanitizer: heap-buffer-overflow /work/harness.c:23:21 in
LLVMFuzzerTestOneInput
0x502000000053 is located 0 bytes after 3-byte region
```

```
[0x502000000050,0x502000000053)
Test unit written to ./crash-0eb8e4ed029b774d80f2b66408203801cb982a60
```

The bug is a missing `size > 3` bound check that only manifests at *runtime* for one exact 3-byte input — there's no unsafe-looking API (no `strcpy`, no format string) for a pattern-matcher to key on, just an off-by-one in manual bounds logic. libFuzzer finds it by mutating toward the magic bytes and watching coverage edges light up, which is dynamic discovery, not static. (Build artifacts — `fuzz`, `corpus/`, `crash-*` — are git-ignored and were deleted after the run.)

Task 5 — Scan the project target (40 min) · *Goal:* apply the tools to your term project. *Steps:* run Semgrep + Gitleaks against **NoteVault** (`../../project/starter-app`); also run an SCA scan: `docker run --rm -v "$PWD/../../project/starter-app:/src" aquasec/trivy fs /src`. *Deliverable:* a findings list (tool, file:line/CVE, CWE) — reuse it in your project vuln report.

Result: `docker run --rm -v "$PWD/../../project/starter-app:/src" semgrep/semgrep semgrep --config p/default --config p/owasp-top-ten /src` → `Ran 365 rules on 5 files: 31 findings.` Gitleaks against the same target → `no leaks found`. Trivy fs → multiple HIGH CVEs in pinned dependencies.

Tool	File:Line/CVE	CWE	Note
Semgrep	`app.py:128-130`	CWE-89	Login query string-formats `username`/`password` into SQL
Semgrep	`app.py:178-179`	CWE-89	`/search` string-formats `user` + `term` into SQL
Semgrep	`app.py:83`	CWE-347	`jwt.decode(..., algorithms=["HS256","none"])` accepts the `none` algorithm
Semgrep	`app.py:134`	CWE-798/522	Hardcoded JWT signing secret (`SECRET = "notevault-dev-secret"`)
Semgrep	`app.py:136`	CWE-614	Session cookie set without `secure`/`httponly`/`samesite`
Semgrep	`app.py:68-117`	CWE-327	MD5 used as the password hash
Semgrep	`app.py:202`	CWE-78	`/export` builds a shell command by string-concatenating `fmt` with `shell=True`
Semgrep	`Dockerfile:12`	CWE-250	No `USER` — container runs as root
Trivy	PyJWT 1.7.1 → CVE-2022-29217 (HIGH)	—	Key-confusion vuln, several years behind current
Trivy	Werkzeug 2.0.1 → CVE-2024-34069 etc.	—	Multiple HIGH/MEDIUM CVEs across the pinned range
Trivy	requests 2.25.1, urllib3 1.26.4 → CVE-2023-	—	Outdated pins, known CVEs

Tool	File:Line/CVE	CWE	Note
	32681, CVE-2021-33503 (HIGH)		
Gitleaks	— (no leaks)	—	`SECRET = "notevault-dev-secret"` is a hardcoded-credential *code* finding (Semgrep's job) — Gitleaks looks for high-entropy secret-shaped literals, and this is low-entropy plain text, so it correctly doesn't fire

Task 6 — Build a security CI gate (25 min) · *Goal:* automate the scan (previews Week 15). *Steps:* adapt `../week15-devsecops-pipeline/security-ci.yml` into a workflow that runs Semgrep + Trivy + Gitleaks and **fails on HIGH/CRITICAL**; run it locally (`act`) or commit to your fork and read the Actions log. *Deliverable:* the workflow file + a screenshot of a failing run.

Result: Adapted `security-ci.yml`, scoped to `project/starter-app` only (`notevault-security-ci.yml`, not checked into this repo's own `.github/workflows/` — this repo's `.semgrepignore`/`.gitleaks.toml` deliberately exclude `project/starter-app/` so the shared course CI doesn't stay permanently red on teaching material; this workflow belongs in your own fork). `act` wasn't available locally, so the gate *logic* was verified by running the same commands its gate steps run:

```
$ semgrep scan --config p/default --config p/owasp-top-ten --error
project/starter-app
exit code: 1        # would fail the build, as intended
$ trivy fs --scanners vuln,secret --severity HIGH,CRITICAL --exit-code 1
project/starter-app
exit code: 1        # would fail the build, as intended
```

Task 7 — SAST blind spots (20 min) · *Goal:* see what scanners miss. *Steps:* find one real bug in `vulnerable-repo/app.py` (or NoteVault) that Semgrep did **not** flag, and explain why a pattern-based tool missed it. *Deliverable:* the bug + a 2-sentence explanation.

Result: `GET /api/notes/<int:nid>` in NoteVault (`app.py:161-168`) checks only that *some* user is logged in — it never checks that the requesting user *owns* the note with id `nid`. Any authenticated user can enumerate note IDs and read other users' notes, including admin's note containing a fake "prod db password" (IDOR / broken object-level authorization, CWE-639). Confirmed Semgrep's own output has no finding anywhere near those lines.

Every other finding in this app is *syntactic* — a known-dangerous API used a known-dangerous way. The IDOR has no dangerous API call at all: the query is correctly parameterized and the code "looks safe" line-by-line. The bug is a *missing* comparison between the note's owner and the logged-in user, which requires understanding app-specific business logic — a semantic judgment Semgrep's pattern rules aren't written to make. (Same root cause elsewhere: `/register` at line 114 lets the client supply `role` directly, so `POST /register {"role":"admin",...}` self-grants admin.)

Task 8 — Defend / fix it (10 min) · *Goal:* remediate the planted flaws in `vulnerable-repo/app.py`. *Steps:* rewrite `/user` to use a parameterized query (`?` placeholder); remove `shell=True` and pass an argument list in `/ping`; move both secrets to environment variables; replace `md5` with bcrypt/argon2; set `debug=False`. *Deliverable:* a before/after diff for each fix mapped to its CWE.

> **Result:** Fixed version written to `app-fixed.py` (kept separate from `vulnerable-repo/app.py` — that file has to stay broken for the rest of the class to scan).

CWE	Before	After
CWE-89	`q = "SELECT * FROM users WHERE name = '%s'" % name` `con.execute(q)`	`q = "SELECT * FROM users WHERE name = ?"` `con.execute(q, (name,))`
CWE-78	`subprocess.check_output("ping -c 1 " + host, shell=True)`	`subprocess.check_output(["ping", "-c", "1", host])`
CWE-798	`AWS_SECRET_ACCESS_KEY = "hK8pQ2..."` `DB_PASSWORD = "xQ7mK2..."`	`AWS_SECRET_ACCESS_KEY = os.environ["AWS_SECRET_ACCESS_KEY"]` `DB_PASSWORD = os.environ["DB_PASSWORD"]`
CWE-327	`hashlib.md5(pw.encode()).hexdigest()`	`bcrypt.hashpw(pw.encode(), bcrypt.gensalt())`
CWE-489	`app.run(debug=True)`	`app.run(debug=False)`

Part 4 — Reflection

1. Map two of your findings to their CWE and to the matching OWASP 2025 category.
2. Name a real-world breach caused by a hardcoded/leaked secret or an injection flaw, and what control would have caught it pre-release.
3. Which single tool (SAST vs. secret scanning) gave the highest-value findings on this repo, and why?

Answers

1. `app.py:19` string-built SQL query → CWE-89 → OWASP 2025 **A05 Injection**. `app.py:30` MD5 password hash → CWE-327 → OWASP 2025 **A04 Cryptographic Failures**.
2. Uber's 2016 breach: attackers found AWS credentials hardcoded in a private repository on a contractor's GitHub Enterprise instance and used them to pull an S3 bucket holding ~57 million riders' and drivers' records. A secret scanner running in CI/pre-commit — exactly this week's Task 2 — would have caught the hardcoded AWS key before or at commit time, the same way it caught the planted key in `app.py:11`.
3. Semgrep, on this specific repo: it covers 4 of the 5 planted flaw categories (SQLi, command injection, weak hash, debug flag) versus Gitleaks' 1 category (2 line-items). Per-finding, a leaked AWS key is arguably worse (instant account compromise, no further exploitation needed), but in terms of breadth of real bugs surfaced on this codebase, Semgrep did more work.

Grading rubric (100)

Criterion	Points
Lecture questions (Part 2)	20
Exploitation + evidence (scan output + triage table + screenshots)	40
Defense (remediated `app.py` with before/after diffs)	25
Reflection (CWE/OWASP mapping + breach + tool value)	15

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root).*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
    1. What did you do, and **why did the vulnerability work**?
    2. **Why does your fix actually stop it** — and what could still break it?

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

Answers

1. Asked an AI to fix the SQL injection in `/user`:

```
@app.route("/user")
def user():
    name = request.args.get("name", "")
    con = sqlite3.connect("app.db")
    q = "SELECT * FROM users WHERE name = '%s'" % name.replace("'", "''")
    return str(con.execute(q).fetchall())
```

2. `name.replace("'", "''")` doubles single quotes, which happens to be SQLite's own string-literal escape convention, so this specific query is no longer trivially breakable with a bare `'`. But it's still

string-formatted SQL, not a parameterized query — a fragile, dialect-dependent fix. The same pattern is unsafe against a database/config where backslash escaping is in play, and it does nothing for a future column that doesn't need quotes to break out of (e.g. a numeric comparison). The AI treated "stop this one payload" as equivalent to "fix the vulnerability class."

3. Correct fix (see Task 8 / `app-fixed.py`): `con.execute("SELECT * FROM users WHERE name = ?", (name,))`. This is correct because the driver sends the query and the value over the wire separately — `name` is never parsed as SQL syntax regardless of its content, so there's no escaping convention to get wrong.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Answers

A. The `/user` endpoint takes a `name` value straight from the URL's query string and glues it into a SQL query with Python string formatting instead of a placeholder. If a request sends a name like `x' OR '1'='1`, the quote in the input closes the string literal early, and `OR '1'='1'` turns the `WHERE` clause into something that's always true — so the query returns every row in the `users` table instead of just rows matching that name. The database has no way to tell attacker-supplied syntax apart from the literal value it was supposed to compare against, because both arrived in the same string.

B. Prompt used:

> "In this Flask + sqlite3 route, `name` comes from `request.args` and is spliced into a SQL string with `%` formatting, causing SQL injection (CWE-89): `q = \"SELECT * FROM users WHERE name = '%s'\" % name; con.execute(q)`. Rewrite this to use a parameterized query with `?` placeholders and `execute(query, params)`. Do not use string formatting, string concatenation, or manual quote-escaping anywhere in the fix."

Result: `q = "SELECT * FROM users WHERE name = ?"` / `con.execute(q, (name,))` — matches `app-fixed.py` (Task 8). Verification: feeding `name = "x' OR '1'='1"` through this version, the database treats the whole string as one opaque parameter to compare against the `name` column — no row is named that, so it returns an empty result instead of every row. The exploit fails.